

ARBITER — Project Brief

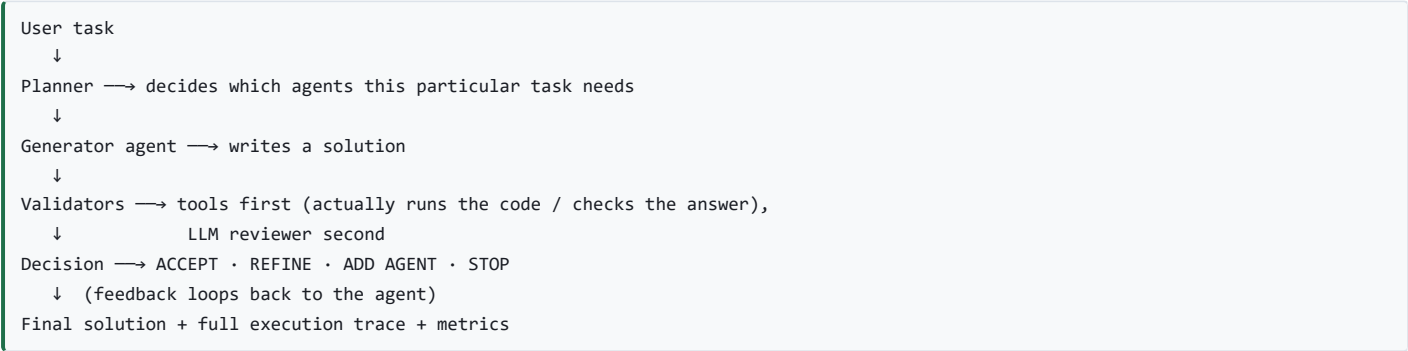
Adaptive Role-Based Iterative Task Execution and Refinement 7th Semester B.Tech CSE Project · SMVDU Katra · 24 September 2026 Gautam Lasgotra (23BCS032) · Chirag Attri (23BCS024) · Aniket Kundal (23BCS015)

1. What we are building

A system where **one agent solves a task, another tests and validates the result, a third solves it again using that feedback, and the cycle repeats until the solution is correct** — which is exactly the architecture proposed in the project review.

One necessary addition: the loop is **bounded**. "Repeat until correct" never terminates on a task the model cannot solve, so the system also stops on iteration, token, call and time limits, on convergence, and when it detects it is repeating itself. Every stop is logged with its reason.

The system accepts a new natural-language task at runtime. There are no pre-written answers and no task-specific hard-coded flows.



2. What makes it research and not just an app

Multi-agent collaboration is well-established (AgentCoder, Self-Refine, Reflexion, ChatDev, MetaGPT). Adaptive agent selection is also already an active subfield with a published survey. **We claim novelty in neither** — claiming it would not survive scrutiny.

The genuine open question comes from a contradiction in the recent literature:

- **Tran & Kiela (2025)** showed that when the token budget is held equal, a single agent matches or beats multi-agent systems on reasoning tasks. Their conclusion: reported multi-agent advantages are "better explained by unaccounted computation" than by architecture.
- Yet **adaptive orchestration papers** (DyFlow, DAAO, AdaptOrch, AgentSpawn, AORCHESTRA) report large gains — and **none of them run a budget-matched comparison**, even though spending compute efficiently is their entire justification.

Our contribution: run that missing test. Does adaptive multi-agent refinement still pay off once the single agent is given the *same* token budget — and does the answer depend on how strong the automatic verifier is?

A supporting result explains why verifier strength should matter: **Huang et al. (ICLR 2024)** found that LLMs cannot reliably self-correct *without external feedback*, and often get worse when they try. So the active ingredient in any refinement loop should be the tool that checks the work, not the agent that criticises it. We test that directly.

This contribution is a measurement, not an invention — which means a **negative result is still a valid result**. That is deliberate: it removes the risk of the project failing if the adaptive approach turns out not to help.

3. Experimental design

Five conditions, one engine, differing only by configuration:

	Condition
A	Single agent, one shot
A+	Single agent given the same token budget as D — <i>the critical baseline</i>
B	Generator ↔ validator loop
C	Fixed pipeline: generator → critic → repair
D	ARBITER — planner selects roles and iterations per task

Three task families, chosen because their verifiers differ in strength — this axis is the experiment:

Family	Verifier	Strength
Code (HumanEval+/MBPP+)	executes unit tests	Strong — knows <i>which</i> test failed
SQL (Spider)	compares query results	Medium — right/wrong only
Math (GSM8K)	exact answer match	Weak — no signal about <i>where</i> it went wrong

Measured: correctness, tests passed, iterations, agents used, LLM calls, tokens, latency, cost, false-accept rate (reviewer approved something the tests reject), and failure category using the MAST taxonomy (NeurIPS 2025). All computed from logged execution traces — nothing is estimated by hand.

4. How it is actually built

An agent is not a complicated object. It is a **role prompt + one LLM call + a typed parse**:

```
def generator(task, router, state, feedback=None):
    prompt = build_prompt(task, feedback)
    resp = router.call(prompt, schema=SOLUTION_SCHEMA, state=state)
    return Solution.model_validate_json(resp.text)
```

The orchestrator is a bounded `while` loop over a state object. Validation for code contains **no LLM at all** — it runs the code in an isolated subprocess and reports which tests failed:

```
subprocess.run([sys.executable, "-I", script], timeout=5, capture_output=True, env=SAFE_ENV)
```

"Adaptive" means the planner returns a different list of roles for different tasks. That is the whole mechanism — kept small so it can be inspected and explained.

Roles are **selected from a fixed library**, never invented at runtime. Runtime invention is unbounded and cannot be measured or reproduced.

Stack: Python · FastAPI · Pydantic · custom ~400-line orchestrator (no heavyweight agent framework, so the contribution stays visible) · Gemini Flash-Lite behind a provider abstraction · SQLite + JSONL traces · deployed on Render free tier.

5. Status and plan

Already working (24 Sep): frozen data contracts, budget governor, response cache, multi-key router, sandboxed execution, graded code validator, math validator, the refinement loop, adaptive planner, resumable experiment runner, 20 passing offline tests. Verified end-to-end: a wrong first attempt is caught by the tests, the feedback is returned, and the second attempt is accepted.

Date	Milestone
29 Sep – 10 Oct	Conditions A, A+, B on the code family; live UI; first results
10 Oct	Internal 50% checkpoint
19–21 Oct	Mid-semester evaluation
11–31 Oct	Adaptive condition D, condition C, SQL + math families
1–10 Nov	Full benchmark sweep, metrics, ablations, deployment, report
10 Nov	Internal 100% (13 days of buffer)

Main risk — API quota. A full sweep is several thousand LLM calls against a free tier. Four controls: a response cache (re-runs cost nothing, and it makes results reproducible), three keys round-robin, a resumable runner that survives quota exhaustion mid-sweep, and a **reserved key used only for the live demo** so evaluation-day quota is always fresh. The interface can also replay a stored trace, so the demonstration works even if the API does not.

6. Work distribution

Member	Responsibility
Gautam Lasgotra (23BCS032)	System architecture, orchestration loop, decision policy, budget governor, provider abstraction, web interface, deployment, integration
Chirag Attri (23BCS024)	Agent role library, prompt design and versioning, structured output schemas, LLM reviewer, model/provider integration
Aniket Kundal (23BCS015)	Benchmark datasets, execution validators, experiment runner, metrics and plots, failure taxonomy, reproducibility

Interfaces between the three modules are frozen so work proceeds in parallel; each member owns a directory and defends that component.

Repository: github.com/igautamlasgotra/arbiter **Key references:** Tran & Kiela (2025) budget-matched comparison · Huang et al., *LLMs Cannot Self-Correct Reasoning Yet*, ICLR 2024 · Cemri et al., *Why Do Multi-Agent LLM Systems Fail?* (MAST), NeurIPS 2025 · AgentCoder (2023) · Self-Refine (2023) · Reflexion (2023)